DAY 5 · CLOUD LANDING ZONES

Week 1 in a new cloud decides the next 5 years.



DEJI AJISHE · DevOps | Multi-Cloud

The 6 foundations →

INFOGRAPHIC 1/3 CLOUD LANDING ZONES

Build a landing zone in this order: foundations first

A landing zone is the foundation every workload inherits: structure, identity, logging, guardrails, network and cost controls, all as code before the first app arrives.

01

Design the hierarchy

Separate platform from workloads, prod from non-prod, and give experiments a sandbox.

Why: these boundaries contain mistakes, scope access and make costs attributable.

```
aws organizations create-organizational-unit --parent-id r-abcd --name Workloads
```

02

Federate identity from one provider

One IdP for every cloud; access through groups, never individual users.

Why: joiners and leavers are handled in one place, for every cloud at once.

```
gcloud resource-manager folders add-iam-policy-binding FOLDER_ID \
  --member=group:platform-admins@example.com --role=roles/viewer
```

03

Centralise audit logs before anything else

Org-wide audit logs flow into a separate, locked-down account or project.

Why: the people being audited must not be able to erase the trail.

```
aws cloudtrail create-trail --name org-trail --s3-bucket-name acme-audit-logs \
  --is-organization-trail --is-multi-region-trail
```

04

Set up break-glass access

Two emergency admin accounts, phishing-resistant MFA, credentials stored offline.

Why: when the IdP is down, you still need a way in, and every use must page someone.

```
Alert rule: any sign-in by a break-glass account → page the on-call engineer
```

✓ Foundations are done when

- **Every account/subscription/project** sits in the hierarchy, none at the root
- **Nobody has a local cloud user**; all access comes from IdP groups
- **Audit logs from every cloud** land in the log archive, read-only for admins
- **Break-glass use** has been tested once and raised an alert

✗ Order mistakes

- Deploying workloads before logging exists: no trail for the first months
- Creating users per cloud "for now": they never get cleaned up
- One shared admin login for the whole team
- Designing hierarchy around the org chart, not around policy and risk

INFOGRAPHIC 2/3 GUARDRAILS, NETWORK, COST

Then make the safe path the default path

05

Apply preventive guardrails at the top

Policies on the root or top folders apply to everything below: deny, don't just warn.

Why: preventing a risky action is cheaper than cleaning up after it.

```
gcloud resource-manager org-policies enable-enforce \
  iam.disableServiceAccountKeyCreation --organization=ORG_ID
```

06

Turn on detective controls

Continuous posture checks catch what guardrails miss, with findings in one place.

Why: too strict a guardrail creates workarounds; detection covers the gap.

```
aws securityhub enable-security-hub --enable-default-standards
```

07

Plan IP ranges and build the network hub

Non-overlapping CIDRs across every cloud and on-prem, then a hub that spokes attach to.

Why: overlapping ranges block peering and VPNs later, and renumbering is painful.

```
10.0.0.0/8 → one /12 per cloud → one /16 per environment → /24 subnets
```

08

Automate vending with budgets and tags

New accounts, subscriptions and projects come from a pipeline, with the baseline built in.

Why: if the safe path is also the fast path, teams stop creating shadow accounts.

```
vend(team="payments", env="prod") → owner + cost-centre tags, budget alert, guardrails,
logging
```

	AWS	Azure	Google Cloud
Preventive	Service control policies (SCPs)	Azure Policy (deny effect)	Organization Policy constraints
Detective	Security Hub, AWS Config	Microsoft Defender for Cloud	Security Command Center
Network hub	Transit Gateway	Hub VNet or Virtual WAN	Shared VPC

Roll out guardrails safely: audit/report-only mode first → sandbox → non-prod → prod. Review what would have been blocked at each stage, and publish an exception process before you enforce.

INFOGRAPHIC 3/3 THE SAME DESIGN IN THREE CLOUDS

One landing zone, three vocabularies

	AWS	Azure	Google Cloud
Hierarchy	Organization → OUs → accounts	Management groups → subscriptions → resource groups	Organization → folders → projects
Identity	IAM Identity Center, federated to your IdP	Microsoft Entra ID, PIM for just-in-time admin	Cloud Identity or Workforce Identity Federation
Audit logs	Organization CloudTrail trail to a log archive account	Activity log diagnostic settings to a central workspace	Aggregated log sink at the organization level
Reference	Control Tower, Security Reference Architecture	Azure landing zones (Cloud Adoption Framework)	Enterprise foundations blueprint
Vending	Account Factory for Terraform (AFT)	Subscription vending	Project factory

✓ Must know

- **Start from the provider's reference**, not a blank page
- **Everything as code:** hierarchy, policies, logging, vending
- **Just-in-time admin** instead of standing privileges
- **Test guardrails in a sandbox** before applying them org-wide
- **Budget alerts** on every sandbox from day one

✗ Pitfalls that cost years

- Everything in one account or subscription
- Audit logs stored where admins can delete them
- Overlapping IP ranges between clouds
- Guardrails so strict teams build workarounds
- Click-ops accounts nobody can reproduce

Phase 1	**Foundations:** hierarchy, identity federation, central audit logs, break-glass (steps 01-04)
Phase 2	**Controls:** preventive guardrails in audit mode, then enforced; detective controls on (05-06)
Phase 3	**Connectivity and scale:** IP plan, network hub, vending pipeline with budgets and tags (07-08)
Phase 4	**First workload:** onboard one real team through vending; fix every manual step they hit

> **Day-one test:** can a new team get a compliant, logged, budgeted environment in under an hour, without a ticket? If not, fix vending first.

WHAT & WHY

What a landing zone is, and why it comes first

A **landing zone** is the pre-built foundation every workload inherits: account structure, identity, networking, logging, security guardrails and governance, all defined as code before the first application arrives.

Without one

One shared account · admin rights "for now" · logs the admins can delete · no cost owner · every team reinvents security

With one

Isolated environments · central SSO · tamper-resistant audit logs · guardrails that apply automatically · new accounts in minutes

Why first: moving live workloads between accounts, subscriptions or projects later is slow, risky and expensive.

REFERENCE ARCHITECTURE

Platform first, workloads underneath

```
AWS Organization
├── Security OU          # Log Archive + Audit / security tooling accounts
├── Infrastructure OU    # network hub, shared services
├── Workloads OU → Prod OU / Non-prod OU → one account per app & env
└── Sandbox OU           # looser guardrails, hard budgets

Azure (Azure landing zones)
Tenant root group → Organization MG
├── Platform → Identity · Management · Connectivity
├── Landing zones → Corp (private) · Online (internet-facing)
├── Sandbox
└── Decommissioned

Google Cloud (enterprise foundations blueprint)
Organization
├── bootstrap / common folders  # CI/CD, logging, shared VPC hosts
└── production / nonproduction / development folders → projects
```

THE 6 DESIGN AREAS

What to design, cloud by cloud

	AWS	Azure	GCP
1 Structure	Organizations + OUs + accounts	Management groups + subscriptions	Folders + projects
2 Identity	IAM Identity Center	Entra ID + PIM	Cloud Identity / Workforce Identity Federation
3 Network	Transit Gateway hub	Hub VNet / Virtual WAN + Azure Firewall	Shared VPC host projects
4 Logging	Organization CloudTrail → Log Archive	Diagnostic settings → Log Analytics	Aggregated org log sinks
5 Guardrails	SCPs + Config rules	Azure Policy	Organization Policy
6 Governance	Tag policies · Budgets	Tags · Cost Management budgets	Labels · Billing budgets

Starting points: AWS Control Tower · Azure landing zones · Google Cloud enterprise foundations blueprint.

THE PROCESS

Building a landing zone, step by step

01 **Capture requirements:** compliance, data residency, regions, team structure, budget owners

02 **Start from the provider's reference** rather than a blank page

03 **Create the hierarchy:** platform vs workloads, prod vs non-prod, sandbox

04 **Centralize identity:** SSO, group-based roles, documented break-glass accounts

05 **Stand up logging and security accounts/projects** before anything else

06 **Build the network hub:** IP plan, hub-and-spoke, controlled egress

07 **Apply guardrails at the top:** preventive policies first, detective second

08 **Automate vending:** new accounts/projects created by pipeline, already compliant

STEP 3 · HIERARCHY

Create the structure from the CLI

```
# AWS: an OU and an account inside the organization
aws organizations create-organizational-unit \
  --parent-id r-abcd --name Workloads
aws organizations create-account --email aws-prod-app@acme.example \
  --account-name prod-app

# Azure: management groups under the organization root
az account management-group create --name mg-platform --parent mg-acme
az account management-group create --name mg-landingzones --parent mg-acme

# GCP: environment folders and a project
gcloud resource-manager folders create --display-name=production \
  --organization=123456789012
gcloud projects create acme-prod-app --folder=FOLDER_ID
```

In practice: run these through Terraform or the reference tooling (next pages) so the structure is versioned and reviewable.

STEP 4 · IDENTITY

Who can do what, from day one

- → **One identity provider** federated into IAM Identity Center, Entra ID and Google Cloud
- → **Group-based access:** people join groups; groups map to roles per environment
- → **No standing admin in prod:** use just-in-time elevation (e.g. Entra PIM) with approval and expiry
- → **Break-glass accounts:** two emergency identities, hardware MFA, excluded from SSO dependencies, alerting on every use
- → **No root/owner use day to day:** lock down AWS root users and Global Administrators
- → **Workloads use workload identity**, never long-lived keys

> **Test it:** can you remove a leaver's access to every cloud in one step? If not, identity isn't centralized yet.

STEP 5 · LOGGING

Org-wide audit logs the auditees can't erase

```
# AWS: one organization trail into the Log Archive account's bucket
aws cloudtrail create-trail --name org-trail --s3-bucket-name acme-org-
logs \
  --is-organization-trail --is-multi-region-trail
aws cloudtrail start-logging --name org-trail

# GCP: aggregated sink for all audit logs across the organization
gcloud logging sinks create org-audit-sink \
  storage.googleapis.com/acme-org-audit-logs \
  --organization=123456789012 --include-children \
  --log-filter='logName:"cloudaudit.googleapis.com"'

# Azure: send the subscription activity log to Log Analytics
az monitor diagnostic-settings subscription create --name to-law \
  --location eastus --workspace <workspace-resource-id> \
  --logs '[{"category":"Administrative","enabled":true}]'
```

Key point: the log destination lives in a separate, locked-down account/project, with retention and deletion protection.

STEP 7 · GUARDRAILS AS CODE

Preventive policies at the top of the tree

AWS SCP: keep the audit trail on and accounts in the org

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ProtectFoundations",
    "Effect": "Deny",
    "Action": [
      "cloudtrail:StopLogging",
      "cloudtrail:DeleteTrail",
      "organizations:LeaveOrganization"
    ],
    "Resource": "*"
  }]
}
```

```
# Azure: built-in "Allowed locations" policy on a management group
az policy assignment create --name allowed-locations \
  --policy e56962a6-4747-49cd-b67b-bf8b01975c4c \
  --scope /providers/Microsoft.Management/managementGroups/mg-acme \
  --params '{"listOfAllowedLocations":{"value":["eastus","westeurope"]}}'

# GCP: block service account key creation organization-wide
gcloud resource-manager org-policies enable-enforce \
  iam.disableServiceAccountKeyCreation --organization=123456789012
```

STEP 6 · NETWORK HUB

Hub-and-spoke with controlled egress

AWS

Transit Gateway in the network account; workload VPCs attach as spokes; centralized egress and inspection VPC

Azure

Hub VNet (or Virtual WAN) with Azure Firewall; spokes peer to the hub; routes force egress through the firewall

GCP

Shared VPC: a host project owns the network; service projects deploy into its subnets

```
# GCP: make a host project and attach a service project
gcloud compute shared-vpc enable acme-net-host
gcloud compute shared-vpc associated-projects add acme-prod-app \
  --host-project=acme-net-host
```

→ **Allocate IP ranges from one plan** across all clouds before creating any network

→ **Private endpoints** for managed services keep traffic off the public internet

INFRASTRUCTURE ORCHESTRATION

Landing zones as code, with account vending

	Reference tooling	Vending pattern
AWS	Control Tower + Account Factory for Terraform (AFT)	Git request → pipeline creates a governed account with baselines
Azure	Azure landing zones (Terraform / Bicep, Azure Verified Modules)	Subscription vending module places subscriptions under the right management group
GCP	Enterprise foundations blueprint (Terraform example foundation)	Project factory creates projects in the right folder with Shared VPC attached

→ **The safe path should be the fast path:** a compliant account in minutes beats a ticket queue

→ **Baselines applied at creation:** logging, guardrails, budgets, network attachment, SSO groups

→ **Everything in Git**: hierarchy, policies and vending requests are reviewed like code

STEP 8 · GOVERNANCE

Ownership and cost from day one

- → **Mandatory tags/labels:** owner, env, cost-center, data-class (lowercase works in all three clouds)
- → **Enforce them:** AWS tag policies, Azure Policy "require a tag", GCP label checks in CI
- → **Budgets with alerts** on every account, subscription and project, sent to the owning team
- → **Sandbox rules:** hard budgets and automatic cleanup for experiments
- → **Decommission path:** a place (e.g. Azure's Decommissioned MG) for retiring workloads safely
- → **Review monthly:** untagged resources, idle spend, accounts with no owner

PITFALLS

8 landing-zone mistakes

→ **Starting with one shared account** and promising to split it later

→ **Logs stored where admins can delete them**

→ **No break-glass plan**, or break-glass accounts nobody monitors

→ **Guardrails so strict** that teams build workarounds

→ **Guardrails so loose** that drift and surprise bills appear

→ **Manual account creation**: every new account slightly different

→ **Overlapping IP ranges** across clouds and environments

→ **Ignoring the provider's reference** and reinventing every decision

NEED-TO-KNOWS

What every cloud engineer should know

- → SCPs set the **maximum** permissions in an AWS account; they don't grant anything
- → Azure Policy and GCP Organization Policy **inherit down** the hierarchy
- → Accounts, subscriptions and projects are the **real security and billing boundaries**
- → Preventive guardrails stop bad changes; **detective** controls (Config, Policy audit, SCC) find them
- → Log destinations belong in a **separate, restricted** account or project
- → Shared VPC lets a central team own the network while app teams own their projects
- → Root users and Global Administrators should be **locked away**, not used daily
- → Landing zones are never finished: **version and evolve** them like any product

CHECKLIST

Landing zone readiness checklist

- ☐ Hierarchy: platform, workloads (prod / non-prod), sandbox, decommissioned
- ☐ Central SSO; group-based roles; just-in-time admin
- ☐ Two monitored break-glass accounts with hardware MFA
- ☐ Org-wide audit logs in a separate, protected account/project
- ☐ Preventive guardrails at the top; detective controls enabled
- ☐ Network hub, IP plan and controlled egress in place
- ☐ Mandatory tags/labels enforced; budgets with alerts everywhere
- ☐ Account/project vending automated and compliant by default
- ☐ Everything defined as code and reviewed in Git

HANDS-ON LAB

Try it: A mini landing zone in a sandbox org

01 In a sandbox organization, create a workloads OU / management group / folder

02 Attach one preventive guardrail (e.g. the SCP on the guardrails page)

03 Confirm audit logs flow to a separate logging account or project

04 Try an action the guardrail blocks and read the denial

```
# AWS example (use the matching Azure/GCP commands from earlier pages)
aws organizations create-organizational-unit --parent-id r-abcd --name
Sandbox-Workloads
aws organizations attach-policy --policy-id p-abcd1234 --target-id ou-
abcd-sandbox
aws cloudtrail describe-trails --query 'trailList[].
{n:Name,org:IsOrganizationTrail}'
```

Teardown: Detach the test policy and remove the test OU/management group/folder afterwards.

FURTHER READING

Go deeper: official references

AWS: Security Reference Architecture
docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/

AWS: What is AWS Control Tower?
docs.aws.amazon.com/controltower/latest/userguide/what-is-control-tower.html

Microsoft: Azure landing zones (Cloud Adoption Framework)
learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/

Google Cloud: Enterprise foundations blueprint
cloud.google.com/architecture/security-foundations

Docs change. Always check the current version before you design.

YOUR TURN

New cloud account, first control you switch on: SSO, audit logs or guardrails?

Share yours in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 5 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a